```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb

from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.cluster import KMeans

import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv('new.csv')
df.head()
```

```
{"type":"dataframe","variable_name":"df"}
```

```
df.shape
```

```
(2240, 29)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 29 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   ID                   2240 non-null   int64  
 1   Year_Birth           2240 non-null   int64  
 2   Education            2240 non-null   object 
 3   Marital_Status       2240 non-null   object 
 4   Income               2216 non-null   float64
 5   Kidhome              2240 non-null   int64  
 6   Teenhome             2240 non-null   int64  
 7   Dt_Customer          2240 non-null   object 
 8   Recency              2240 non-null   int64  
 9   MntWines             2240 non-null   int64  
 10  MntFruits            2240 non-null   int64  
 11  MntMeatProducts      2240 non-null   int64  
 12  MntFishProducts      2240 non-null   int64  
 13  MntSweetProducts     2240 non-null   int64  
 14  MntGoldProds         2240 non-null   int64  
 15  NumDealsPurchases    2240 non-null   int64  
 16  NumWebPurchases      2240 non-null   int64  
 17  NumCatalogPurchases  2240 non-null   int64  
 18  NumStorePurchases    2240 non-null   int64  
 19  NumWebVisitsMonth    2240 non-null   int64  
 20  AcceptedCmp3         2240 non-null   int64  
 21  AcceptedCmp4         2240 non-null   int64  
 22  AcceptedCmp5         2240 non-null   int64  
 23  AcceptedCmp1         2240 non-null   int64  
```

```
 24  AcceptedCmp2          2240 non-null   int64
 25  Complain              2240 non-null   int64
 26  Z_CostContact         2240 non-null   int64
 27  Z_Revenue             2240 non-null   int64
 28  Response              2240 non-null   int64
dtypes: float64(1), int64(25), object(3)
memory usage: 507.6+ KB
```

```
df.describe().T
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 26,\n  \"fields\": [\n{\n      \"column\": \"count\",\n      \"properties\": {\n\"dtype\": \"number\",\n        \"std\": 4.706787243316415,\n\"min\": 2216.0,\n        \"max\": 2240.0,\n\"num_unique_values\": 2,\n        \"samples\": [\n          2216.0,\n2240.0\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\":\"mean\",\n      \"properties\": {\n        \"dtype\": \"number\",\n\"std\": 10245.54250883089,\n        \"min\": 0.009375,\n\"max\": 52247.25135379061,\n        \"num_unique_values\": 25,\n\"samples\": [\n          166.95,\n          5.316517857142857\n],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"std\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 4945.88598281448,\n\"min\": 0.0,\n        \"max\": 25173.07666090141,\n\"num_unique_values\": 25,\n        \"samples\": [\n225.71537251175434,\n          2.426645009547285\n        ],\n\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"min\",\n      \"properties\": {\n\"dtype\": \"number\",\n        \"std\": 492.65479410419465,\n\"min\": 0.0,\n        \"max\": 1893.0,\n\"num_unique_values\": 5,\n        \"samples\": [\n          1893.0,\n11.0\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\":\"25%\",\n      \"properties\": {\n        \"dtype\": \"number\",\n\"std\": 6916.682939569983,\n        \"min\": 0.0,\n        \"max\":35303.0,\n        \"num_unique_values\": 12,\n        \"samples\": [\n2.0,\n          9.0\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\":\"50%\",\n      \"properties\": {\n        \"dtype\": \"number\",\n\"std\": 10077.79628484552,\n        \"min\": 0.0,\n        \"max\":51381.5,\n        \"num_unique_values\": 16,\n        \"samples\": [\n5458.5,\n          1970.0\n        ],\n        \"semantic_type\":\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n\"column\": \"75%\",\n      \"properties\": {\n        \"dtype\":\"number\",\n        \"std\": 13453.114094184806,\n        \"min\":0.0,\n        \"max\": 68522.0,\n        \"num_unique_values\": 17,\n\"samples\": [\n          8427.75,\n          1977.0\n        ],\n\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"max\",\n      \"properties\": {\n
```

```
\"dtype\": \"number\",\n        \"std\": 130623.70202867234,\n
\"min\": 1.0,\n        \"max\": 666666.0,\n
\"num_unique_values\": 19,\n        \"samples\": [\n
11191.0,\n          1493.0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe"}
```

```
# df['Acceptance'] = df['Acceptance'].str.replace('Accepted', '')

for col in df.columns:
      temp = df[col].isnull().sum()
      if temp > 0:
            print(f'Column {col} contains {temp} null values.')
```

```
Column Income contains 24 null values.
```

```
df = df.dropna()
print("Total missing values are:", len(df))
```

```
Total missing values are: 2216
```

```
df.nunique()
```

```
ID                     2216
Year_Birth               59
Education                 5
Marital_Status            8
Income                 1974
Kidhome                   3
Teenhome                  3
Dt_Customer             662
Recency                 100
MntWines                776
MntFruits               158
MntMeatProducts         554
MntFishProducts         182
MntSweetProducts        176
MntGoldProds            212
NumDealsPurchases        15
NumWebPurchases          15
NumCatalogPurchases      14
NumStorePurchases        14
NumWebVisitsMonth        16
AcceptedCmp3              2
AcceptedCmp4              2
AcceptedCmp5              2
AcceptedCmp1              2
AcceptedCmp2              2
Complain                  2
Z_CostContact             1
Z_Revenue                 1
```

```
Response                 2
dtype: int64
```

```
parts = df["Dt_Customer"].str.split("-", n=3, expand=True)
df["day"] = parts[0].astype('int')
df["month"] = parts[1].astype('int')
df["year"] = parts[2].astype('int')
```

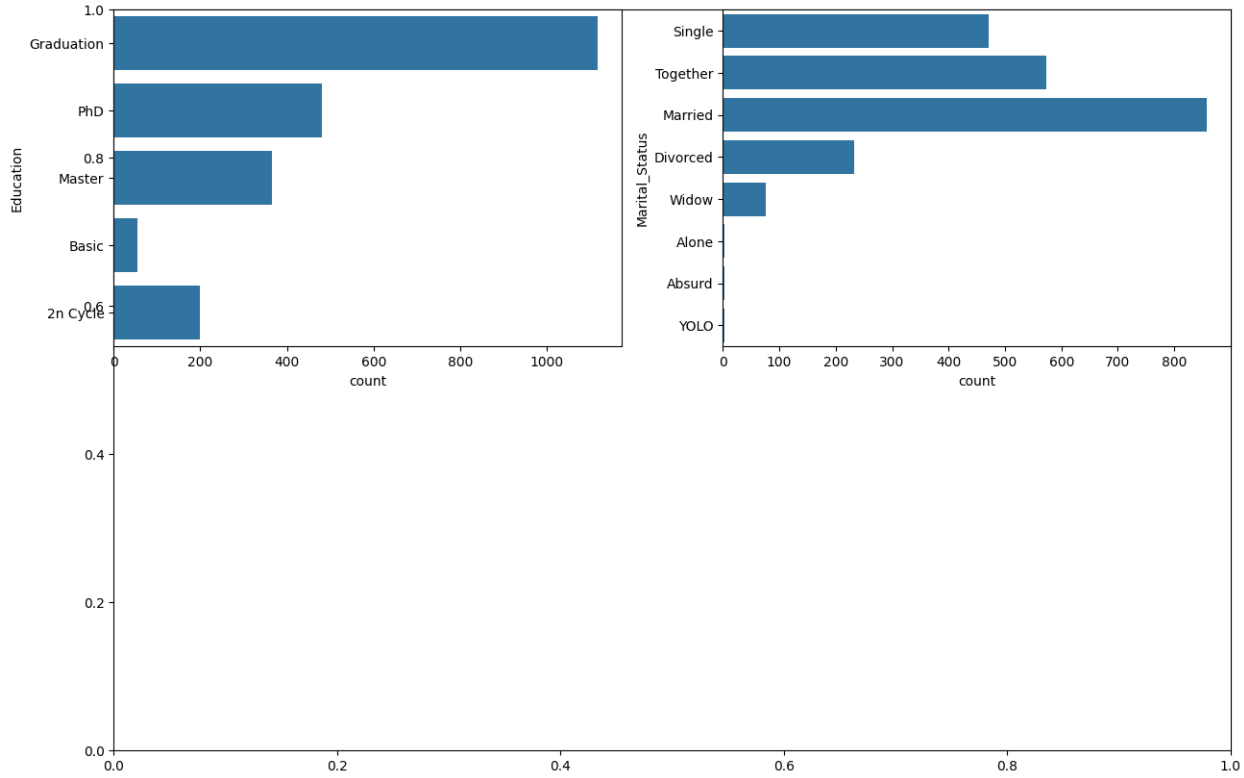
```
df.drop(['Z_CostContact', 'Z_Revenue', 'Dt_Customer'],
           axis=1,
           inplace=True)
```

```
floats, objects = [], []
for col in df.columns:
     if df[col].dtype == object:
          objects.append(col)
     elif df[col].dtype == float:
          floats.append(col)
```

```
print(objects)
print(floats)
```

```
['Education', 'Marital_Status']
['Income']
```

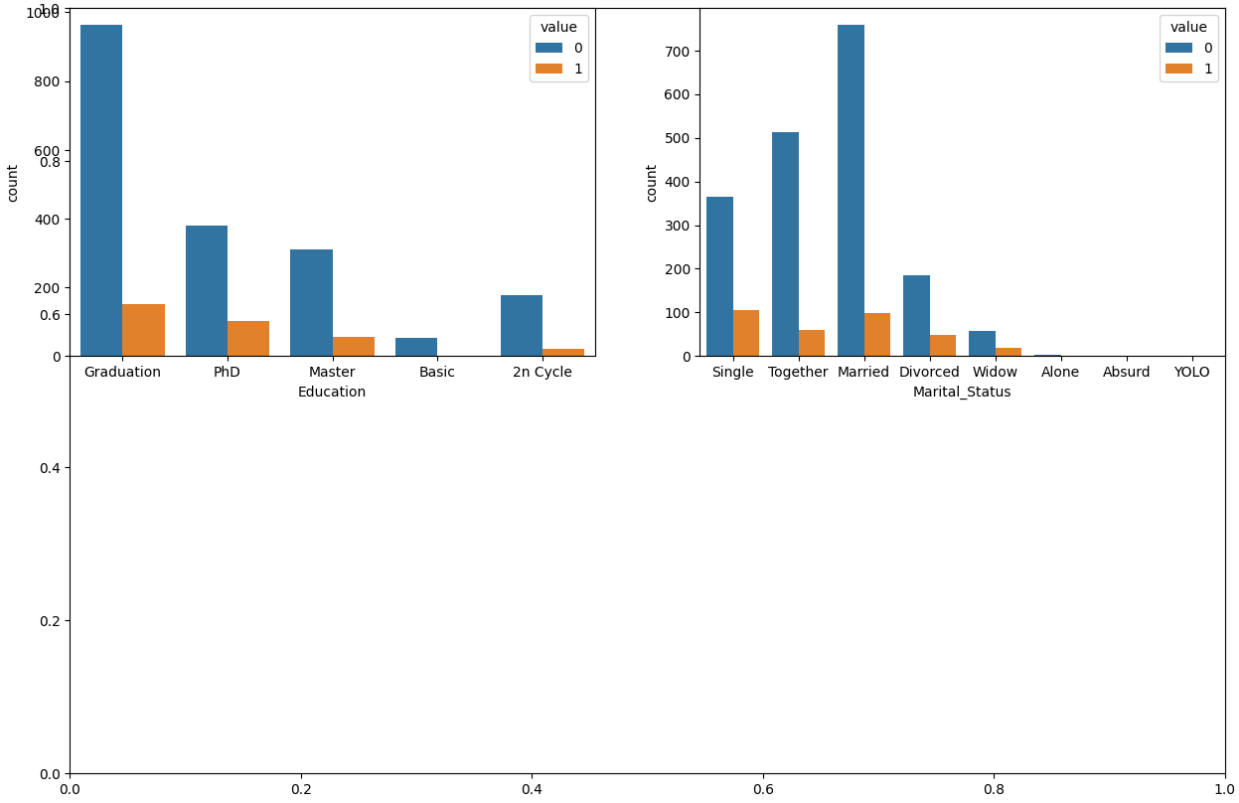
```
plt.subplots(figsize=(15, 10))
for i, col in enumerate(objects):
     plt.subplot(2, 2, i + 1)
     sb.countplot(df[col])
plt.show()
```




```
df['Marital_Status'].value_counts()
```

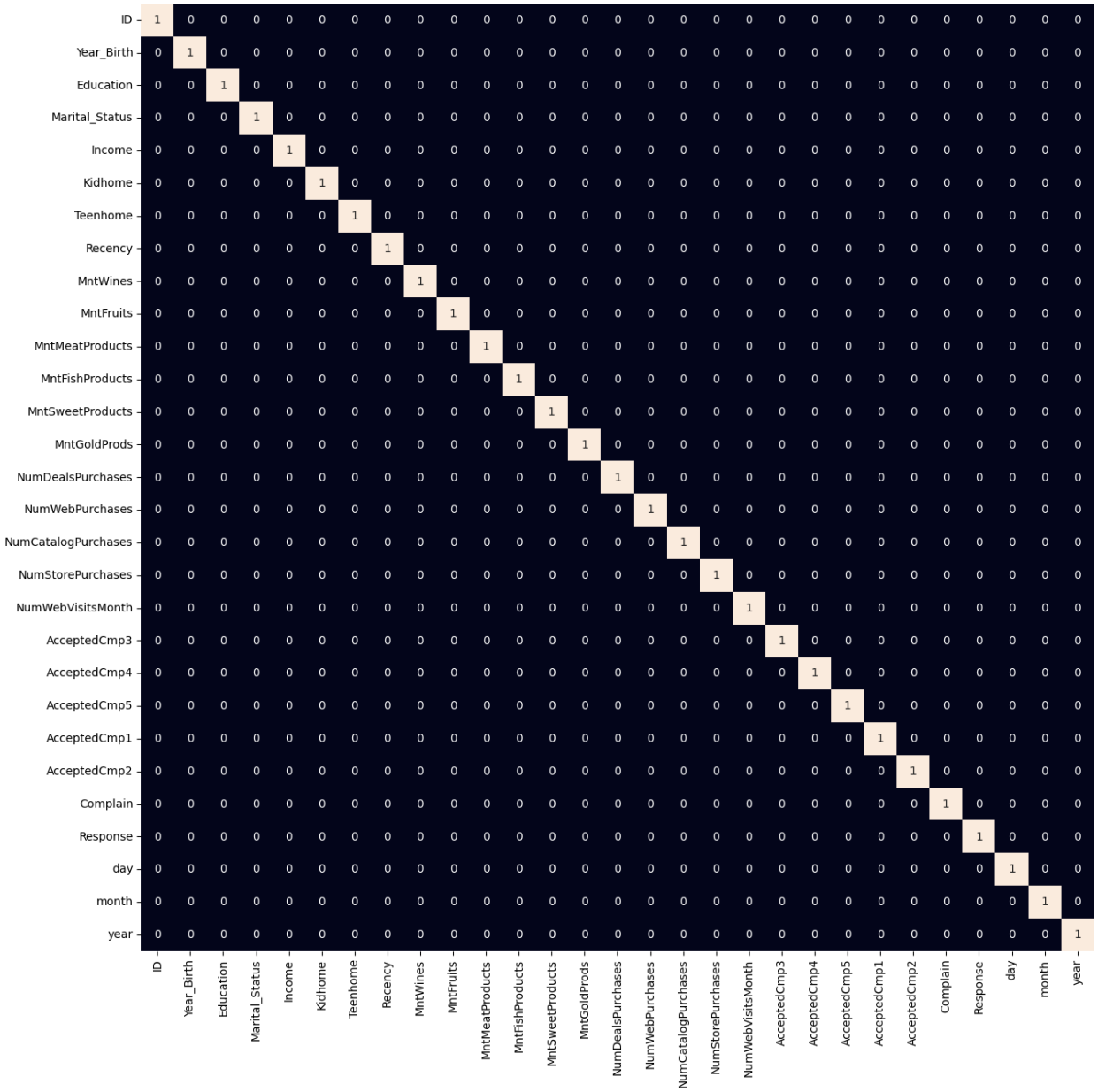
```
Marital_Status
Married     857
Together    573
Single      471
Divorced    232
Widow        76
Alone         3
Absurd        2
YOLO          2
Name: count, dtype: int64
```

```
plt.subplots(figsize=(15, 10))
for i, col in enumerate(objects):
    plt.subplot(2, 2, i + 1)
    # Use melt to transform the data to long form
    df_melted = df.melt(id_vars=[col], value_vars=['Response'],
var_name='hue')
    sb.countplot(x=col, hue='value', data=df_melted)
plt.show()
```



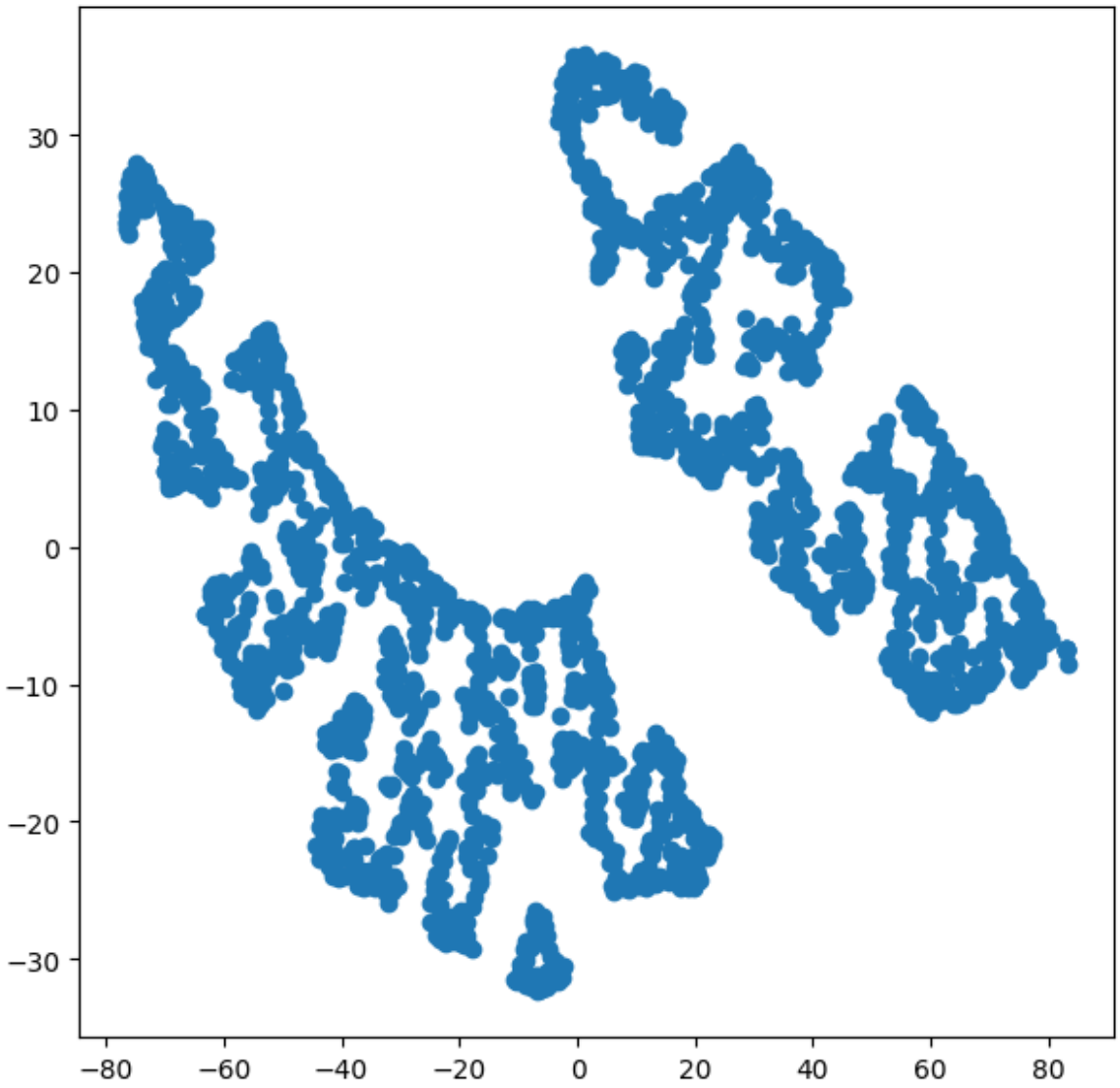

```
for col in df.columns:
     if df[col].dtype == object:
          le = LabelEncoder()
          df[col] = le.fit_transform(df[col])

plt.figure(figsize=(15, 15))
sb.heatmap(df.corr() > 0.8, annot=True, cbar=False)
plt.show()
```



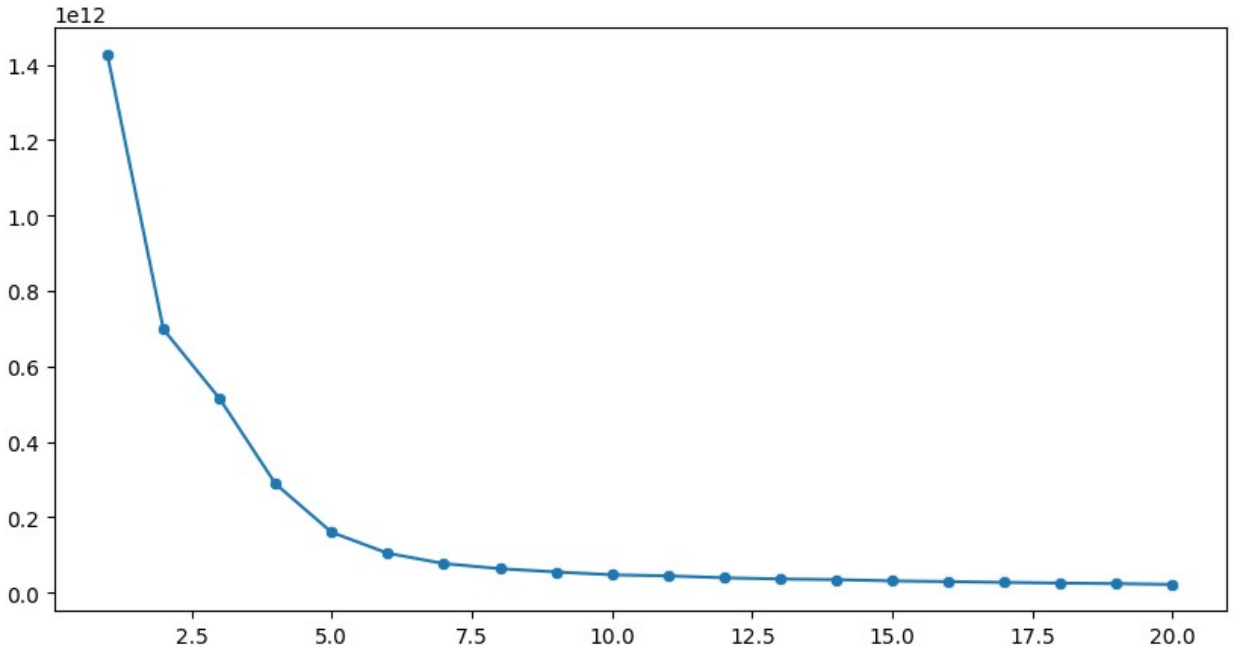

```
scaler = StandardScaler()
data = scaler.fit_transform(df)

from sklearn.manifold import TSNE
model = TSNE(n_components=2, random_state=0)
tsne_data = model.fit_transform(df)
plt.figure(figsize=(7, 7))
plt.scatter(tsne_data[:, 0], tsne_data[:, 1])
plt.show()
```



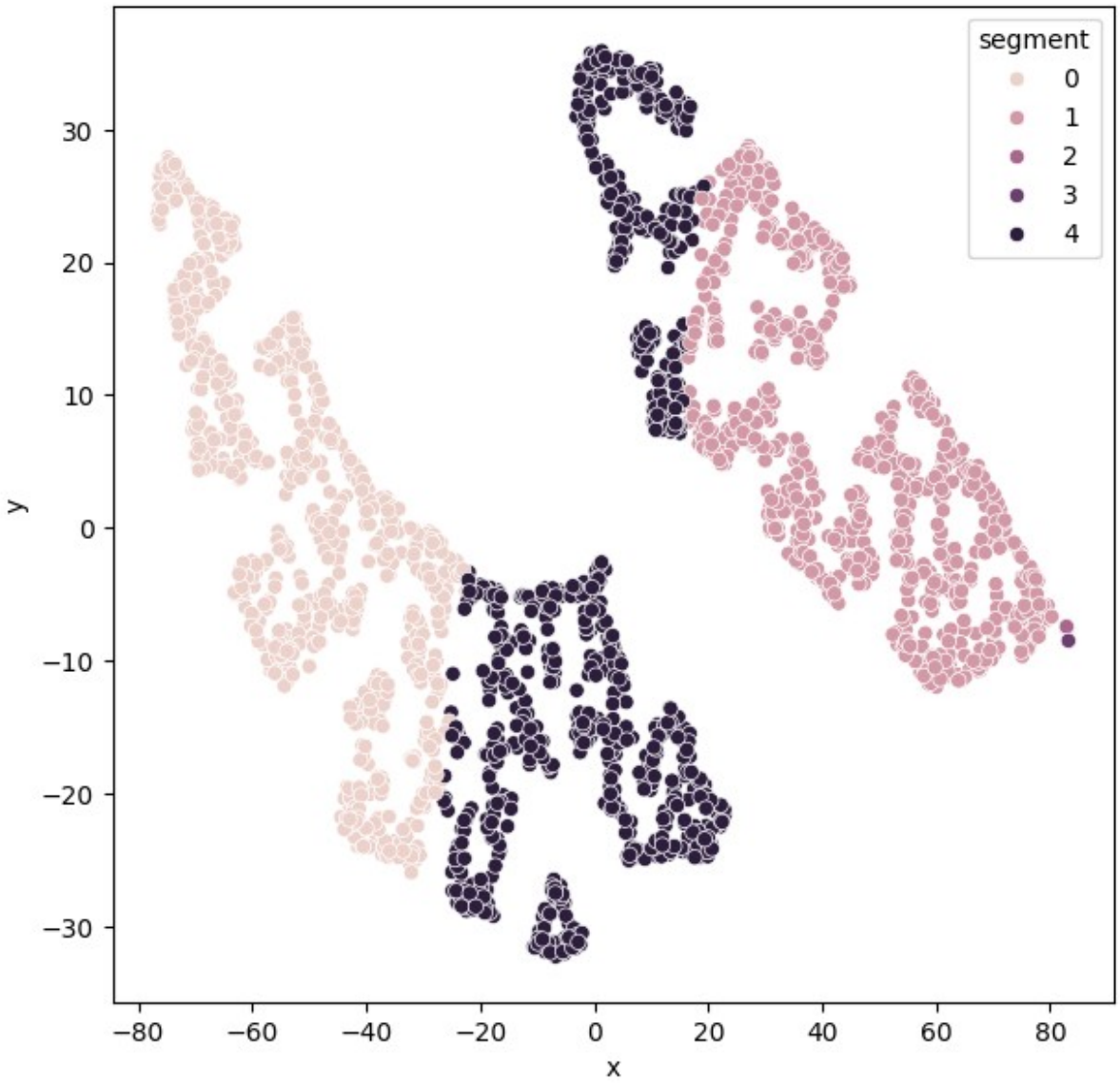

```
error = []
for n_clusters in range(1, 21):
      model = KMeans(init='k-means++',
                     n_clusters=n_clusters,
                     max_iter=500,
                     random_state=22)
      model.fit(df)
      error.append(model.inertia_)

plt.figure(figsize=(10, 5))
sb.lineplot(x=range(1, 21), y=error)
sb.scatterplot(x=range(1, 21), y=error)
plt.show()
```




```
# create clustering model with optimal k=5
model = KMeans(init='k-means++',
                n_clusters=5,
                max_iter=500,
                random_state=22)
segments = model.fit_predict(df)

plt.figure(figsize=(7, 7))
# Create a DataFrame with the tsne_data and segments
df_tsne = pd.DataFrame({'x': tsne_data[:, 0], 'y': tsne_data[:, 1],
'segment': segments})
# Use the DataFrame in the scatterplot function
sb.scatterplot(x='x', y='y', hue='segment', data=df_tsne)
plt.show()
```


segment
0
1
2
3
4
30
20
10
0
−10
−20
−30
y
−80
−60
−40
−20
0
20
40
60
80
x